Ejercicio 5 – Defensa de QTZ2

CC2008 – Programación Orientada a Objetos · Universidad del Valle de Guatemala

Fabio Calvo · Carné: 262256

1. Descripción del problema

Se construye el prototipo de un simulador estilo Defensa de Torres para gestionar los recursos del satélite Quetzal-2 (QTZ2). El operador administra una misión formada por módulos de tres tipos: **módulos de vuelo** (cámaras o sensores en órbita que recolectan datos científicos), **módulos de tierra** (antenas en el campus que descargan esos datos) y **módulos de energía** (paneles solares que mantienen el sistema encendido).

Todos los módulos comparten ID, nombre, salud, estado (activo o inactivo) y costo de construcción, pero responden de forma distinta al pulso del reloj del simulador, `procesarCiclo()`. Los módulos se influyen entre sí mediante tres recursos globales: la energía disponible, los datos en órbita (pendientes de descarga) y los datos descargados. La energía alimenta a los módulos de vuelo y de tierra, y los datos fluyen de vuelo a tierra, por lo que **el orden de participación en el ciclo afecta el resultado**.

2. Análisis

2.1 Entidades y datos

- **Módulo (común):** ID, nombre, salud, activo, costo de construcción.
- **Módulo de vuelo:** instrumento (cámara o sensor), MB recolectados por ciclo, consumo de energía por ciclo, ciclos acumulados y total histórico recolectado.
- **Módulo de tierra:** capacidad de descarga por ciclo, energía por descarga, total descargado y estación.
- **Módulo de energía:** energía generada por ciclo, ciclos acumulados y total histórico generado.
- **Recursos de la misión:** energía disponible, datos en órbita y datos descargados.

2.2 Reglas de negocio

- **RN-01.** Un módulo inactivo no opera: no consume, no produce y no incrementa sus ciclos acumulados.
- **RN-02 (energía).** Un módulo de energía activo produce su cantidad fija por ciclo; la energía queda disponible de inmediato para los demás módulos y se suma a su total histórico generado.
- **RN-03 (vuelo).** Para operar debe estar activo y la energía disponible debe cubrir su consumo. Entonces consume esa energía, recolecta la cantidad completa de MB (no hay recolecciones parciales), la deja en órbita y suma a su total histórico y a sus ciclos acumulados. Si no puede operar, no consume ni recolecta y se informa el motivo.
- **RN-04 (tierra).** Para descargar debe estar activo, tener energía suficiente y haber datos pendientes en órbita. Descarga el mínimo entre su capacidad y los datos en órbita, y consume **toda** su energía por descarga aunque descargue menos que su capacidad. Los datos descargados

dejan de estar pendientes y se suman al total del módulo y al de la misión. Si alguna condición falla, no descarga ni consume y se informa la causa.

- **RN-05.** Los recursos se actualizan inmediatamente después de la participación de cada módulo; cada módulo usa lo que dejó el anterior.
- **RN-06.** Los módulos participan en el orden de la carga inicial. Un ciclo nuevo parte de los recursos resultantes del anterior; los valores no se reinician.
- **RN-07.** Las capacidades por ciclo son constantes desde el inicio de la misión.
- **RN-08.** No se modelan baterías. Los ciclos acumulados cuentan solo los ciclos en que el módulo realmente operó.

2.3 Requisitos funcionales

- **RF-01 Carga inicial.** Al iniciar, el programa puebla una única lista polimórfica con 11 módulos (más de los 10 requeridos) de los tres tipos y fija los recursos iniciales, emulando una misión en progreso.
- **RF-02 Listar.** Muestra todos los módulos con sus características.
- **RF-03 Buscar.** Busca un módulo por ID o por nombre y muestra toda su información.
- **RF-04 Catálogo.** Muestra los módulos ordenados por costo de construcción.
- **RF-05 Avanzar ciclo.** Todos los módulos, en el orden de la carga inicial, tienen una oportunidad de operar. Se informa el resultado de cada uno y la causa cuando no opera, y al final se muestra el estado de los recursos.
- **RF-06 Resumen de comunicaciones.** Muestra el total de módulos de tierra y cuántos están activos, la capacidad total de descarga por ciclo de los activos, el total histórico descargado, y el nombre, ID y estación del módulo de tierra con mayor descarga histórica (todos, si hay empate).

2.4 Restricciones

- Programa en Java con menú de consola.
- Una única lista polimórfica de módulos, con al menos 10 elementos de los tres tipos.
- Diseño con patrón MVC, herencia, polimorfismo y sobreescritura de métodos.
- Buenas prácticas: `toString()` (y `equals()`/`hashCode()` donde sirva), encapsulación con visibilidad y getters, y documentación interna.
- Entregables: este PDF con análisis y diseño (incluye el diagrama UML 1.4) y un repositorio de GitHub con los archivos `.java`.

2.5 Supuestos de diseño

El enunciado deja abiertos varios puntos; estas son las decisiones tomadas:

- **ID:** se genera automáticamente. Es un prefijo de tres letras según el tipo (ENR, VUE, TIE) seguido de un entero aleatorio entre 0 y 9999, único gracias a un conjunto estático de números ya usados.
- **Salud:** valor numérico (`double`) de carácter informativo. No interviene en las reglas de operación, porque el enunciado solo condiciona la operación al estado activo y a los recursos.
- **Estado activo:** todo módulo se crea activo; la carga inicial desactiva algunos con `desactivar()`. El menú no incluye una opción para cambiarlo.

- **Estación:** atributo de tipo `String` propio del módulo de tierra, necesario para el resumen (RF-06).
- **Unidades:** energía y MB se manejan como `double`.
- **Búsqueda:** por ID con coincidencia exacta y por nombre con coincidencia exacta. Si no hay resultados, la vista informa que no se encontraron módulos.
- **Catálogo:** orden ascendente por costo. Se ordena una copia; la lista original conserva el orden de la carga, del cual depende el ciclo.
- **Resultado de un módulo:** `procesarCiclo()` devuelve un objeto `Respuesta` (éxito, mensaje y datos opcionales) en lugar de imprimir, de modo que el modelo no depende de la consola.
- **Recursos compartidos:** existe un único objeto `RecursosMision`, cuya referencia recibe cada módulo en su constructor.
- **Total descargado del resumen:** es el acumulado de la misión. Coincide con la suma de los módulos de tierra porque toda descarga pasa por uno de ellos y la carga inicial es coherente (sección 4).

3. Diseño

3.1 Jerarquía de herencia

```
Modulo (abstracta)
├── ModuloEnergia
└── ModuloConsumidor (abstracta)
    ├── ModuloVuelo
    └── ModuloTierra
```

`Modulo` concentra lo común (ID, nombre, salud, estado, costo, referencia a los recursos) y declara el comportamiento polimórfico `procesarCiclo()`. Los módulos de vuelo y de tierra siguen el mismo protocolo para operar: estar activos, tener energía suficiente, cumplir una condición adicional y entonces operar. Ese protocolo se escribe una sola vez en la clase abstracta intermedia `ModuloConsumidor` (patrón *Template Method*), lo que evita duplicar validaciones y mensajes. `ModuloEnergia` no consume energía, por lo que hereda directamente de `Modulo`.

Complementan el modelo el enumerado `Instrumento` (CAMARA, SENSOR) y el registro `Respuesta<T>` (éxito, mensaje, datos opcionales).

3.2 Especificación de las clases

Modulo (abstracta)

- Atributos: `-IDS: Set<Integer>` {static}, `-id: String`, `-nombre: String`, `-salud: double`, `-activo: boolean`, `-costo: double`, `#recursos: RecursosMision`.
- Métodos: `-generarId(): int` {static}, `#Modulo(nombre, salud, costo, recursos)`, `+procesarCiclo(): Respuesta<?>` {abstract}, `+getPrefijoId(): String` {abstract}, getters (`getId`, `getNombre`, `getSalud`, `getCosto`), `+isActivo()`, `+isInactivo()`, `+activar()`, `+desactivar()`, `+equals(Object)`, `+hashCode()` (ambos por `id`) y `+toString()`.

ModuloEnergia (extiende Modulo)

- Atributos: `-energiaPorCiclo: double`, `-energiaTotalGenerada: double`, `-ciclos: int`.
- Métodos: `+procesarCiclo(): Respuesta<Double>`, `+getPrefijoId()` (ENR), `+toString()`.

ModuloConsumidor (abstracta, extiende Modulo)

- Métodos: `+procesarCiclo(): Respuesta<Void>` (método plantilla), `#getEnergiaRequerida(): double` {abstract}, `#operar(RecursosMision): String` {abstract}, `#motivoBloqueoAdicional(RecursosMision): Optional<String>` (gancho; por defecto vacío).

ModuloVuelo (extiende ModuloConsumidor)

- Atributos: `-instrumento: Instrumento`, `-mbPorCiclo: double`, `-energiaPorCiclo: double` (consumo), `-ciclos: int`, `-mbTotalRecolectados: double`.
- Métodos: `#getEnergiaRequerida()`, `#operar(RecursosMision)`, `+getPrefijoId()` (VUE), `+toString()`.

ModuloTierra (extiende ModuloConsumidor)

- Atributos: `-capacidadDescarga: double`, `-energiaPorDescarga: double`, `-totalDescargado: double`, `-estacion: String`.
- Métodos: `#getEnergiaRequerida()`, `#operar(RecursosMision)`, `#motivoBloqueoAdicional(RecursosMision)`, `+getPrefijoId()` (TIE), `+getCapacidadDescarga()`, `+getTotalDescargado()`, `+getEstacion()`, `+toString()`.

RecursosMision

- Atributos: `-energiaDisponible`, `-datosEnOrbita`, `-datosDescargados` (todos `double`).
- Métodos: `+agregarEnergia(double)`, `+consumirEnergia(double)`, `+registrarRecoleccion(double)`, `+registrarDescarga(double)` y los tres getters. Los módulos no modifican los números directamente: usan operaciones con significado que validan que la cantidad sea positiva.

Mision

- Atributos: `-modulos: List<Modulo>`, `-recursos: RecursosMision`.
- Métodos: `+agregarModulo(Modulo)`, `+getModulos(): Stream<Modulo>` (copia), `+buscarPorId(String): Optional<Modulo>`, `+buscarPorNombre(String): Stream<Modulo>`, `+catalogoPorCosto(): Stream<Modulo>`, `+ciclo(): List<Respuesta<?>>`, `+modulosTierra(): int`, `+modulosTierraActivos(): int`, `+capacidadActiva(): double`, `+modulosTierraMejores(): List<ModuloTierra>`, `+totalDescargado(): double`, `+getRecursos(): RecursosMision`.

CargaInicial

- `+construirInicial(): Mision` {static} y `-inactivo(T): T` {static, auxiliar}.

VistaConsola

- Atributo: `-scanner: Scanner`.
- Métodos: `mostrarMenu`, `mostrarMenuBusqueda`, `leerOpcion(min, max)`, `leerTexto(mensaje)`, `mostrarMensaje`, `mostrarError`, `mostrarModulos`, `mostrarCatalogo`, `mostrarResultadosCiclo`, `mostrarRecursos`, `mostrarResumenComunicaciones`.

ControladorMision

- Atributos: `-mision: Mision`, `-vista: VistaConsola`.
- Métodos: `+iniciar(): boolean` (muestra el menú, atiende una opción y devuelve falso al salir) y privados `listar`, `buscar`, `catalogo`, `avanzarCiclo`, `resumen`.

Main: `+main(String[])` crea la misión, la vista y el controlador, y repite `iniciar()` hasta que devuelve falso.

3.3 Polimorfismo y sobreescritura

Método	Declarado en	Implementado o sobrescrito en
`procesarCiclo()`	`Modulo` (abstracto)	`ModuloEnergia` (genera energía) y `ModuloConsumidor` (método plantilla de vuelo y tierra)
`getPrefijoId()`	`Modulo` (abstracto)	`ModuloEnergia` (ENR), `ModuloVuelo` (VUE), `ModuloTierra` (TIE)
`getEnergiaRequerida()`	`ModuloConsumidor` (abstracto)	`ModuloVuelo` (consumo por ciclo), `ModuloTierra` (energía por descarga)
`operar(RecursosMision)`	`ModuloConsumidor` (abstracto)	`ModuloVuelo` (recolecta), `ModuloTierra` (descarga)
`motivoBloqueoAdicional(RecursosMision)`	`ModuloConsumidor` (concreto, por defecto sin motivo)	`ModuloTierra` (sin datos en órbita)
`toString()`	`Modulo`	`ModuloEnergia`, `ModuloVuelo`, `ModuloTierra` (agregan sus atributos sobre `super.toString()`)
`equals()` / `hashCode()`	`Modulo` (por `id`)	Heredados por todas las subclases

- **Enlace dinámico:** `Mision.ciclo()` recorre la lista `List<Modulo>` y llama a `procesarCiclo()` sin conocer el tipo concreto de cada módulo. Cada objeto responde según su clase: el panel genera energía, la cámara consume y recolecta, y la antena consume y descarga.
- **Parámetros polimórficos:** los comportamientos que varían reciben la abstracción `RecursosMision` (`operar` y `motivoBloqueoAdicional`), de modo que cada subclase actúa sobre el mismo estado compartido sin conocer a las demás.
- **Retornos covariantes:** `Modulo.procesarCiclo()` devuelve `Respuesta<?>`; `ModuloEnergia` lo refina a `Respuesta<Double>` y `ModuloConsumidor` a `Respuesta<Void>`.

- **Uso acotado de `instanceof`:** solo lo usan las consultas del resumen de comunicaciones en `Mision` (RF-06), porque esas estadísticas son específicas de los módulos de tierra y la lista es única. El ciclo no usa comprobaciones de tipo.

3.4 Patrón MVC y separación de responsabilidades

Capa	Clases	Responsabilidad
Modelo	`Modulo`, `ModuloEnergia`, `ModuloConsumidor`, `ModuloVuelo`, `ModuloTierra`, `Instrumento`, `Respuesta`, `RecursosMision`, `Mision`, `CargaInicial`	Reglas de negocio, estado de la misión y datos iniciales. No realiza entrada ni salida por consola.
Vista	`VistaConsola`	Única clase con `Scanner` y `System.out`. Recibe los datos ya calculados y los presenta; no modifica el modelo.
Controlador	`ControladorMision`	Interpreta la opción del menú, pide el trabajo al modelo y entrega el resultado a la vista. No contiene reglas de negocio ni imprime.

`Main` solo compone los objetos. El modelo no conoce a la vista ni al controlador; la vista no conoce al controlador; el controlador conoce a ambos.

3.5 Principios de diseño aplicados

- **Responsabilidad única:** cada clase tiene una razón de cambio; por ejemplo, `RecursosMision` solo administra los tres recursos y `Mision` solo administra la lista y sus consultas.
- **Abierto/cerrado:** un nuevo tipo de módulo se agrega con una nueva subclase, sin modificar `Mision.ciclo()` ni el controlador.
- **Sustitución de Liskov:** cualquier subclase puede usarse donde se espera un `Modulo` sin alterar el funcionamiento del ciclo.
- **Inversión de dependencias:** `Mision` depende de la abstracción `Modulo`, no de las clases concretas.
- **No repetir código:** el protocolo común de vuelo y tierra vive solo en `ModuloConsumidor`.
- **Encapsulación:** atributos privados con getters y operaciones con significado en `RecursosMision`, que valida sus entradas. `Mision.getModulos()` devuelve una copia de la lista.

3.6 Flujo de «Avanzar un ciclo»

1. El controlador recibe la opción 4 y llama a `mision.ciclo()`.
2. `Mision` recorre la lista en el orden de carga y llama a `procesarCiclo()` de cada módulo.

3. Cada módulo valida sus condiciones y actualiza de inmediato los `RecursosMision`:
    - **Energía:** si está inactivo, informa que no está activo; si no, agrega su energía y actualiza sus acumulados.
    - **Vuelo y tierra (`ModuloConsumidor`):** si está inactivo, informa; si la energía disponible es menor que `getEnergiaRequerida()`, informa; si `motivoBloqueoAdicional()` trae un motivo, lo informa; en caso contrario ejecuta `operar()` y devuelve el detalle de lo realizado.
4. `Mision` devuelve la lista de `Respuesta`, una por módulo y en el mismo orden.
5. El controlador pasa a la vista los módulos, las respuestas y los recursos; la vista muestra el resultado de cada módulo y el estado final de los recursos.

4. Carga inicial y caso de prueba

Recursos iniciales: **25** de energía disponible, **210 MB** en órbita y **830 MB** descargados. El orden de la tabla es el orden de participación en el ciclo.

#	Tipo	Nombre	Estado	Salud	Costo	Datos propios
1	Energía	Panel Solar Norte	Activo	100	450	20 por ciclo, 18 ciclos
2	Vuelo	Cámara Óptica Alfa	Activo	95	820	Cámara, 25 MB, consumo 8, 14 ciclos
3	Tierra	Antena Campus UVG	Activo	90	600	Capacidad 40, energía 6, descargado 320, Campus UVG
4	Vuelo	Sensor Magnetómetro	Activo	88	540	Sensor, 15 MB, consumo 5, 18 ciclos
5	Energía	Panel Solar Sur	Activo	92	380	15 por ciclo, 20 ciclos
6	Tierra	Antena Quetzaltenango	Activo	85	560	Capacidad 30, energía 5, descargado 100,

#	Tipo	Nombre	Estado	Salud	Costo	Datos propios
						Estación Quetzaltenango
7	Vuelo	Cámara Infrarroja Beta	Inactivo	40	910	Cámara, 30 MB, consumo 10, 10 ciclos
8	Tierra	Antena Petén	Inactivo	55	480	Capacidad 25, energía 4, descargado 90, Estación Petén
9	Energía	Panel Solar Auxiliar	Inactivo	60	250	10 por ciclo, 16 ciclos
10	Vuelo	Sensor de Radiación	Activo	97	430	Sensor, 10 MB, consumo 4, 12 ciclos
11	Tierra	Antena Cobán	Activo	80	390	Capacidad 20, energía 4, descargado 320, Estación Cobán

Coherencia de los datos. Los módulos de vuelo recolectaron históricamente 25×14 + 15×18 + 30×10 + 10×12 = 1040 MB. Los módulos de tierra descargaron 320 + 100 + 90 + 320 = 830 MB, igual que los datos descargados de la misión. La diferencia, 210 MB, es lo que está pendiente en órbita.

Resultado esperado del primer ciclo:

#	Módulo	Operación	Energía	En órbita	Descargados
1	Panel Solar Norte	Genera 20	45	210	830

#	Módulo	Operación	Energía	En órbita	Descargados
2	Cámara Óptica Alfa	Consume 8, recolecta 25 MB	37	235	830
3	Antena Campus UVG	Consume 6, descarga 40 MB	31	195	870
4	Sensor Magnetómetro	Consume 5, recolecta 15 MB	26	210	870
5	Panel Solar Sur	Genera 15	41	210	870
6	Antena Quetzaltenango	Consume 5, descarga 30 MB	36	180	900
7	Cámara Infrarroja Beta	No opera: inactivo	36	180	900
8	Antena Petén	No opera: inactivo	36	180	900
9	Panel Solar Auxiliar	No opera: inactivo	36	180	900
10	Sensor de Radiación	Consume 4, recolecta 10 MB	32	190	900
11	Antena Cobán	Consume 4, descarga 20 MB	28	170	920

Resultado esperado del resumen de comunicaciones (estado inicial): 4 módulos de tierra, 3 activos; capacidad total de descarga por ciclo de los activos: 90 MB (40 + 30 + 20); total histórico descargado: 830 MB; mayor descarga histórica: Antena Campus UVG y Antena Cobán, empatadas con 320 MB.

5. Diagrama de clases UML

